《数字图像处理》实验报告一

姓名：顾凯琪	学号：226001624	班级：计算机 2 班

1. 实验名称：MATLAB 数字图像基本操作

2. 实验内容与目标

（1）掌握 Matlab 界面和功能；

（2）使用 Matlab 进行数字图像的基本操作；

3. 实验任务

（1）将灰度图像 woman.jpg 转换成索引图像，颜色分别为 gray(128)、gray(16)

（2）将灰度图像 snowflakes.png 转换成索引图像。利用函数 grayslice 将一幅灰度图像转换成索引图像

（3）将真彩色图像 flower.jpg 转换成二值图像

（4）将一幅真彩色图像 flower.jpg 转换成灰度图像

（5）显示一幅二值图像，并将原始的二值图像取反后显示

（6）在 MATLAB 的命令行中输入 inf=imfinfo('woman.jpg')，查询文件 woman.jpg 的信息

（7）利用 imadd 函数给一幅图像 rice.png 加上一个常数

（8）两幅图像 rice.png，cameraman.tif 相加

（9）两幅图像相减

（10）将两幅图像相乘

（11）敲入基本代码，并调试 Matlab 程序

（12）撰写实验报告。

4. 实验方法与过程简要介绍（请填入下面空白处）

1. 灰度图像转换为索引图像

实验目标：

将灰度图像 **woman.jpg** 转换为索引图像，并分别使用 **128** 级和 **16** 级的灰度索引显示。

实验方法：

[1]. 读取 **woman.jpg**，该图像为灰度图像。

[2]. 使用 **gray2ind** 将其转换为索引图像，并分别设置颜色级数为 **128** 和 **16**。

[3]. 依次显示原始灰度图像及转换后的索引图像，观察不同颜色级别对图像的影响。

实验现象与分析：

- 在 **128** 级灰度索引下，图像细节较为丰富，接近原始灰度图像。
- 在 **16** 级灰度索引下，图像的颜色层次减少，表现出较明显的色阶变化，损失了一定细节，但仍能保持整体轮廓。

2. 使用 **grayslice** 进行索引图像转换

实验目标：

将 **snowflakes.png** 由灰度图像转换为 **16** 级索引图像，并使用 **jet(16)** 颜色映射方式显示。

实验方法：

[1]. 读取 **snowflakes.png**，该图像为灰度图像。

[2]. 使用 **grayslice** 将图像均匀量化为 **16** 个灰度等级，并生成索引图像。

[3]. 使用 **jet(16)** 颜色映射方式显示索引图像，以增强视觉效果。

实验现象与分析：

- 由于 **grayslice** 采用均匀量化，图像中灰度值被分为 **16** 个等级，表现出较为明显的颜色分区。
- 使用 **jet(16)** 进行颜色映射后，图像呈现不同颜色区域，使得各级灰度的对比更加明显。

3. 真彩色图像转换为二值图像

实验目标：

利用 **im2bw** 函数将 **flower.jpg** 转换为二值图像，并观察不同阈值对结果的影响。

实验方法：

[1]. 读取 **flower.jpg**，该图像为 **RGB** 彩色图像。

[2]. 设定归一化阈值为 **0.4**，使用 **im2bw** 进行二值化处理。

[3]. 显示二值化图像，并观察二值化效果。

实验现象与分析：

- 归一化阈值设为 **0.4** 时，像素值大于 **0.4** 的部分变为白色，低于 **0.4** 的部分变为黑色，生成二值图像。
- 调整阈值会影响二值化效果，例如较小的阈值会使更多区域变为白色，而较大的阈值会使图像变得更加黑暗。

4. 真彩色图像转换为灰度图像

实验目标：

利用 **rgb2gray** 函数将 **flower.jpg** 从真彩色图像转换为灰度图像。

实验方法：

[1]. 读取 **flower.jpg**，该图像为 **RGB** 彩色图像。
[2]. 使用 **rgb2gray** 进行灰度转换。
[3]. 显示灰度图像，并与原始彩色图像对比观察。
实验现象与分析：

- 经过 **rgb2gray** 处理后，彩色图像变为单通道灰度图像，颜色信息被去除，仅保留亮度信息。
- 由于灰度转换使用了加权公式 **Y = 0.2989*R + 0.5870*G + 0.1140*B**，红色、绿色、蓝色的权重不同，因此转换后的灰度图像并非简单的黑白处理，而是根据人眼感知调整的亮度信息。

5. 二值图像取反
实验目标：
对 **circles.jpg** 进行二值图像的取反处理。

实验方法：

[1]. 读取 **circles.jpg**，该图像为二值图像。
[2]. 使用 **~** 进行取反操作。
[3]. 显示取反前后图像，并进行对比观察。
实验现象与分析：

- 取反后的图像黑白颠倒，原来是黑色的部分变为白色，白色的部分变为黑色。
- 该操作可用于背景与前景的互换，适用于某些图像分割和特征提取任务。

6. 查询图像文件信息
实验目标：
使用 **imfinfo** 查询 **woman.jpg** 图像文件的信息。

实验方法：

[1]. 使用 **imfinfo** 获取 **woman.jpg** 的详细信息。
[2]. 观察返回的信息，包括文件路径、大小、格式、分辨率、颜色类型等。
实验现象与分析：

- 返回的信息包括文件格式（**JPEG**）、文件大小（**29058** 字节）、图像宽高（**500**

×500)、位深度（8 位)、颜色类型（灰度图像)、编码方式（Huffman）等。

- 该方法可用于检查图像属性，以便进行后续处理。

7. 图像加上常数

实验目标:

使用 **imadd** 让 **rice.png** 图像像素值整体增加 **50**。

实验方法:

[1]. 读取 **rice.png**，该图像为灰度图像。

[2]. 使用 **imadd** 对所有像素值增加 **50**。

[3]. 显示原始图像和增加后的图像，并进行对比。

实验现象与分析:

- 增加常数后，整个图像变亮，但不会超过最大灰度值 **255**。
- 该方法可用于图像增强，提高亮度。

8. 两幅图像相加

实验目标:

使用 **imadd** 对 **rice.png** 和 **cameraman.tif** 进行图像相加。

实验方法:

[1]. 读取两幅灰度图像。

[2]. 使用 **imadd** 进行加法运算，并显示相加后的图像。

实验现象与分析:

- 结果图像为两幅图像的亮度叠加，适用于多重曝光和图像融合。

9. 两幅图像相减

实验目标:

使用 **imsubtract** 进行图像减法运算，实现背景去除、边缘增强等效果。

实验方法:

[1]. 读取 **rice.png**，该图像是一张灰度图像。

[2]. 通过 **strel** 函数生成结构元素（圆盘形 **disk(15)**)。

[3]. 使用 **imopen** 进行形态学开运算，去除小的前景区域，得到背景图像。

[4]. 通过 **imsubtract** 计算原图像与背景图像的差值，去除背景，仅保留亮度较

高的目标区域。

[5]. 显示原始图像、背景图像和处理后的图像。

实验现象与分析：

- 由于 **rice.png** 是米粒图像，其背景亮度相对均匀，米粒较亮，背景较暗。
- 通过 **imopen** 进行背景提取后，再利用 **imsubtract** 进行相减操作，可有效去除背景，使米粒更加突出。
- 该方法常用于医学图像处理（去除干扰背景）、目标检测等领域。

10. 图像相乘

实验目标：

使用 **immultiply** 进行图像乘法运算，以增强特定区域或实现遮罩处理。

实验方法：

[1]. 读取 **rice.png**，该图像为灰度图像。

[2]. 生成一个与原图像尺寸相同的二值掩码（例如 **mask = im2bw(rice, 0.5)**），用于选择图像的特定部分。

[3]. 使用 **immultiply** 进行图像相乘，得到经过掩码处理的图像。

[4]. 显示原始图像、掩码图像和相乘后的结果图像。

实验现象与分析：

- **immultiply** 的作用类似于图像的“按位相乘”，即在掩码图像的白色（**1**）部分保持原值，黑色（**0**）部分变为黑色。
- 该操作常用于区域提取、**ROI**（感兴趣区域）选择，例如在医学影像处理中提取特定组织区域。
- 如果直接使用数值常数（如 **immultiply(rice, 0.5)**），则可以实现整体亮度调整，使图像整体变暗。

5. 实验程序与程序注释（请填入下面空白处）

实验一

```
% 读取灰度图像 woman.png
l = imread('1/woman.png');

% 创建一个包含 3 个子图的图形窗口
figure;

% 第一幅子图：显示原始图像
subplot(1, 3, 1);  % 1 行 3 列，当前为第 1 个子图
```

```
imshow(I);  % 显示原始图像
title('原始图像');

% 第二幅子图：转换为 128 色索引图像
[I1, map128] = gray2ind(I, 128);  % 使用 gray2ind 函数生成 128 色索引图像
subplot(1, 3, 2);  % 1 行 3 列，当前为第 2 个子图
imshow(I1, map128);  % 显示 128 色索引图像
title('128 色索引图像');

% 第三幅子图：转换为 16 色索引图像
[I2, map16] = gray2ind(I, 16);  % 使用 gray2ind 函数生成 16 色索引图像
subplot(1, 3, 3);  % 1 行 3 列，当前为第 3 个子图
imshow(I2, map16);  % 显示 16 色索引图像
title('16 色索引图像');
```

实验二

```
% 1. 读取灰度图像 snowflakes.png
I = imread('snowflakes.png');  % 读取灰度图像

% 2. 使用 grayslice 函数将图像量化为 16 个等级
X = grayslice(I, 16);  % 将灰度图像量化为 16 个等级

% 3. 显示原始图像和量化后的图像
figure;

% 第一幅子图：显示原始图像
subplot(1, 2, 1);  % 1 行 2 列，当前为第 1 个子图
imshow(I);  % 显示原始图像
title('原始图像');

% 第二幅子图：显示量化后的图像
subplot(1, 2, 2);  % 1 行 2 列，当前为第 2 个子图
imshow(X, jet(16));  % 显示量化后的图像，使用 jet 颜色映射
title('量化后的 16 级图像');
```

实验三

```
% 读取真彩色图像
RGB = imread('1/flower.jpg');

% 使用 im2bw 将图像转换为二值图像，阈值设为 0.4
BW = im2bw(RGB, 0.4);
```

```
% 显示原图和二值图像
figure;
subplot(1, 2, 1);
imshow(RGB);
title('原始真彩色图像');

subplot(1, 2, 2);
imshow(BW);
title('二值图像');
```

实验四

```
% 读取真彩色图像
RGB = imread('1/flower.jpg');  % 使用 imread 函数读取名为'flower.jpg'的真彩色图像

% 将真彩色图像转换为灰度图像
grayImage = rgb2gray(RGB);  % 使用 rgb2gray 函数将 RGB 图像转换为灰度图像

% 显示原图和灰度图像
figure;  % 创建新图形窗口
subplot(1, 2, 1);  % 将当前窗口分为 1 行 2 列，选择第一个位置
imshow(RGB);  % 显示原始真彩色图像
title('原始真彩色图像');  % 为图像添加标题

subplot(1, 2, 2);  % 选择第二个位置
imshow(grayImage);  % 显示转换后的灰度图像
title('灰度图像');  % 为图像添加标题
```

实验五

```
% 读取二值图像
BW = imread('1/circles.png');  % 使用 imread 函数读取二值图像'circles.png'
BW = im2bw(BW, 0.4);  % 将图像转换为二值图像，阈值为 0.4

% 创建图形窗口并使用 subplot 将两个图像显示在一个窗口中
figure;  % 创建新图形窗口

% 显示原始二值图像
subplot(1, 2, 1);  % 将窗口分为 1 行 2 列，选择第一个位置
```

```
imshow(BW);  % 显示原始的二值图像
title('原始二值图像');  % 为图像添加标题

% 显示取反后的二值图像
subplot(1, 2, 2);  % 选择第二个位置
imshow(~BW);  % 使用逻辑取反操作(~)显示取反后的图像
title('取反后的图像');  % 为图像添加标题
```

实验六

```
info = imfinfo('1/woman.png') % 使用 imfinfo 函数查询图像'woman.jpg'的详细信息
```

实验七

```
% 读取图像
I = imread('1/rice.png');  % 使用 imread 函数读取图像'rice.png'

% 给图像加上常数 50
Iplus50 = imadd(I, 50);  % 使用 imadd 函数将图像与常数 50 相加

% 显示原始图像和加法后的图像
figure;  % 创建新图形窗口
subplot(1, 2, 1);  % 将窗口分为 1 行 2 列，选择第一个位置
imshow(I);  % 显示原始图像
title('原始图像');  % 为图像添加标题

subplot(1, 2, 2);  % 选择第二个位置
imshow(Iplus50);  % 显示加法后的图像
title('加法后的图像');  % 为图像添加标题
```

实验八

```
% 读取两幅图像
I1 = imread('1/rice.png');  % 使用 imread 函数读取第一幅图像'rice.png'
I2 = imread('cameraman.tif');  % 使用 imread 函数读取第二幅图像'cameraman.tif'

% 两幅图像相加
I_add = imadd(I1, I2);  % 使用 imadd 函数将两幅图像相加

% 显示两幅图像和相加后的图像
```

```
figure;  % 创建新图形窗口
subplot(1, 3, 1);  % 将窗口分为 1 行 3 列，选择第一个位置
imshow(I1);  % 显示第一幅图像
title('第一幅图像');  % 为图像添加标题

subplot(1, 3, 2);  % 选择第二个位置
imshow(I2);  % 显示第二幅图像
title('第二幅图像');  % 为图像添加标题

subplot(1, 3, 3);  % 选择第三个位置
imshow(I_add);  % 显示相加后的图像
title('相加后的图像');  % 为图像添加标题
```

实验九

```
% 读取图像
I = imread('rice.png');  % 使用 imread 函数读取图像'rice.png'

% 创建背景图像
background = imopen(I, strel('disk', 15));  % 使用 imopen 函数对图像进行形态学开运算，创建背景图像

% 图像相减
I_sub = imsubtract(I, background);  % 使用 imsubtract 函数将原图像与背景图像相减

% 显示原图和相减后的图像
figure;  % 创建新图形窗口
subplot(1, 2, 1);  % 将窗口分为 1 行 2 列，选择第一个位置
imshow(I);  % 显示原始图像
title('原始图像');  % 为图像添加标题

subplot(1, 2, 2);  % 选择第二个位置
imshow(I_sub, []);  % 显示相减后的图像，使用[]来自动调整显示范围
title('相减后的图像');  % 为图像添加标题
```

实验十

```
I = imread('1/moon.png');

I16 = uint16(I);  % 转换为 uint16
J = immultiply(I16, I16);  % 进行 uint16 乘法
```

```
% 归一化到 0-255 并转换回 uint8
J = uint8(mat2gray(J) * 255);

% 使用 imshowpair 进行对比
figure;
imshowpair(I, J, 'montage');
title('Original Image (Left) vs Multiplied Image (Right)');
```

6. 实验结果与总结（请填入下面空白处）

实验一：**MATLAB** 数字图像基本操作

实验结果

在此实验中，我们使用了 **woman.jpg** 图像，首先将其转换为 **128** 色和 **16** 色的索引图像。通过 **gray2ind** 函数，图像被成功地转换为不同色彩深度的版本。在显示的结果中，**128** 色索引图像相比于 **16** 色图像，更能保留图像的细节和颜色，而 **16** 色图像在视觉效果上显得较为粗糙。通过这两个转换结果，我们清楚地看到色彩深度对图像细节的影响。



实验总结

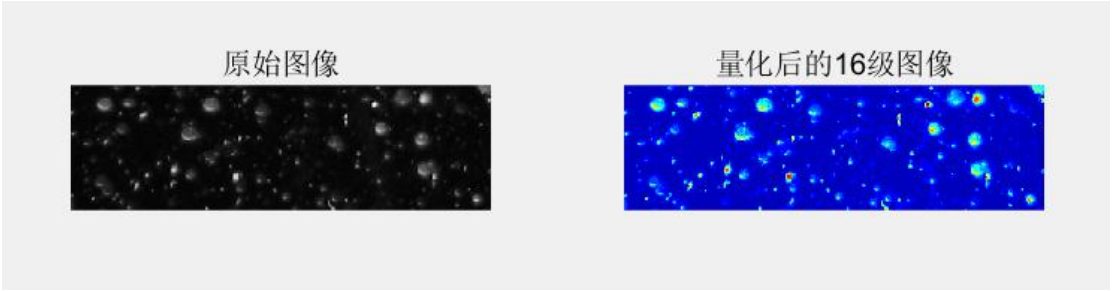
本实验帮助我们掌握了如何通过 **gray2ind** 函数将灰度图像转换为索引图像，并展示了色彩深度对图像质量的影响。较低的色彩深度虽然能有效减少图像的存储

需求，但也会导致图像细节的丢失。在实际应用中，选择合适的色彩深度需要在图像质量和存储/计算效率之间做出平衡。

实验二：图像量化

实验结果

在将 **snowflakes.png** 图像转换为索引图像时，我们使用了 **grayslice** 函数，将灰度图像量化为 **16** 个等级。经过量化处理后，图像的灰度级别显著减少，显示出较明显的色调变化和细节丢失，尤其是色彩的过渡变得不那么平滑。



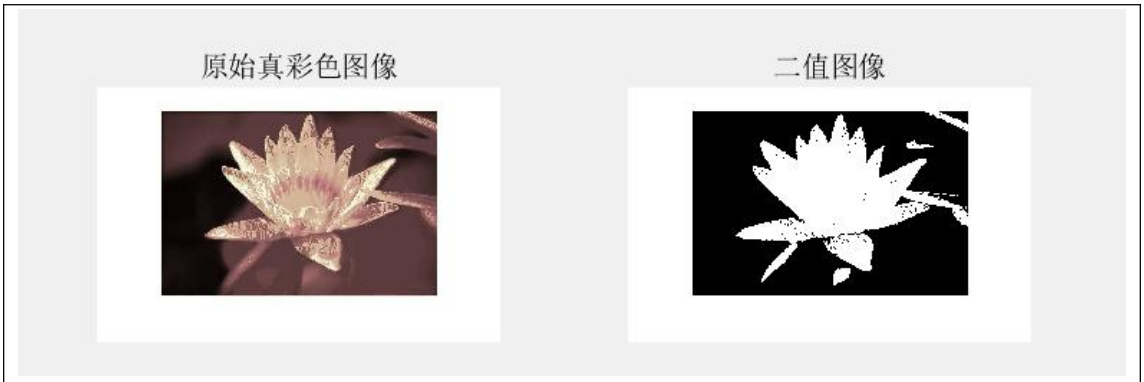
实验总结

本实验展示了图像量化的过程，通过 **grayslice** 函数将灰度图像量化为较少的灰度级别后，图像的细节丢失变得明显。图像量化是一种常见的压缩方法，适用于需要降低计算负担或存储需求的场景，但会损失一些图像的细节信息。

实验三：图像二值化

实验结果

通过 **im2bw** 函数将 **flower.jpg** 转换为二值图像后，图像的像素值根据阈值（**0.4**）被分类为 **0** 或 **1**。这种转换显著简化了图像，使得图像仅由两种颜色组成，具有更高的对比度和清晰的边界。



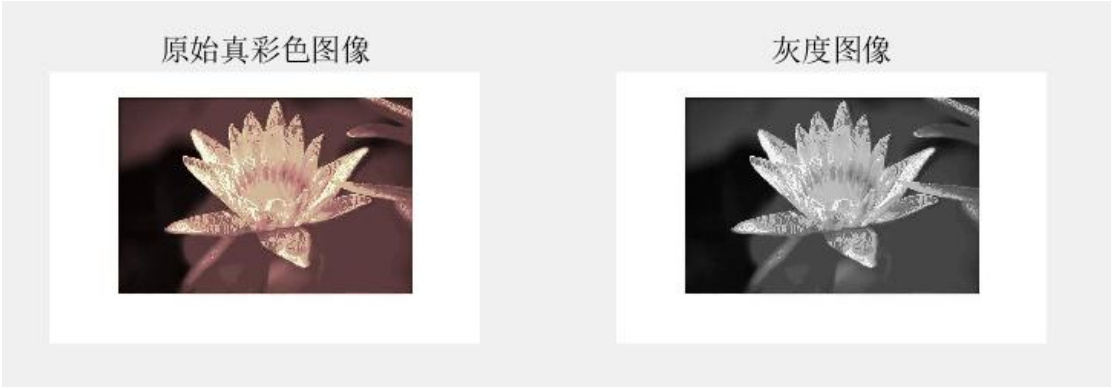
实验总结

本实验通过 **im2bw** 函数实现了图像的二值化处理，成功地将彩色图像转换为二值图像，突出了图像的形状和轮廓。在图像处理中，二值化是一种常见的图像分割技术，能够帮助提取图像的主要结构或特征。

实验四：真彩色图像转灰度图像

实验结果

通过 **rgb2gray** 函数将 **flower.jpg** 这幅彩色图像转换为灰度图像后，图像失去了所有的颜色信息，只保留了亮度信息。虽然转换后图像的细节有所保留，但颜色特征完全消失，图像呈现出灰度的不同层次。



实验总结

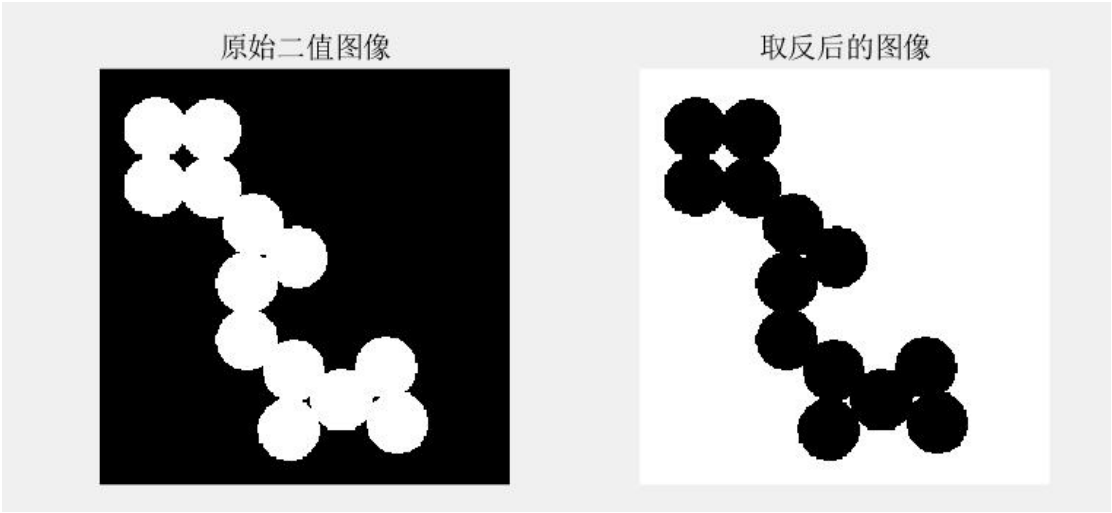
此实验演示了将真彩色图像转换为灰度图像的方法，主要用于图像处理中的图像简化。在某些应用中，我们不需要保留颜色信息，只需要图像的亮度信息，这时

转换为灰度图像将有助于减少计算量和提高处理效率。

实验五：图像取反

实验结果

成功加载并取反了二值图像 **circles.jpg**，通过取反操作后，图像中的黑色部分变为白色，白色部分变为黑色。取反的图像可以清晰地展示反转前后的对比，显示出图像的反转效果。



实验总结

通过 **imshow(~BW)** 实现图像取反的操作，在图像处理中，取反常用于图像增强、反色显示或进行对比度增强。取反操作简单且高效，可以突出图像中的特定特征。

实验六：图像信息查询

实验结果

使用 **imfinfo** 函数查询了图像 **woman.jpg** 的详细信息，包括文件大小、格式、宽度、高度、颜色类型等元数据。图像的宽度为 **500** 像素，高度为 **500** 像素，且为灰度图像格式。

命令行窗口

包含以下字段的 struct:

```
              Filename: 'C:\Users\asus\Desktop\x\试卷卷卷子\数字图像\实验\实验一\1\woman.png'
           FileModDate: '22-Mar-2022 19:57:06'
              FileSize: 102535
                Format: 'png'
         FormatVersion: []
                 Width: 349
                Height: 351
              BitDepth: 24
             ColorType: 'truecolor'
       FormatSignature: [137 80 78 71 13 10 26 10]
              Colormap: []
             Histogram: []
        InterlaceType: 'none'
          Transparency: 'none'
   SimpleTransparencyData: []
       BackgroundColor: []
       RenderingIntent: 'perceptual'
        Chromaticities: [0.3127 0.3290 0.6400 0.3300 0.3000 0.6000 0.1500 0.0600]
                 Gamma: 0.4546
           XResolution: []
           YResolution: []
        ResolutionUnit: []
               XOffset: []
               YOffset: []
            OffsetUnit: []
       SignificantBits: []
          ImageModTime: []
                 Title: []
                Author: []
           Description: []
             Copyright: []
```

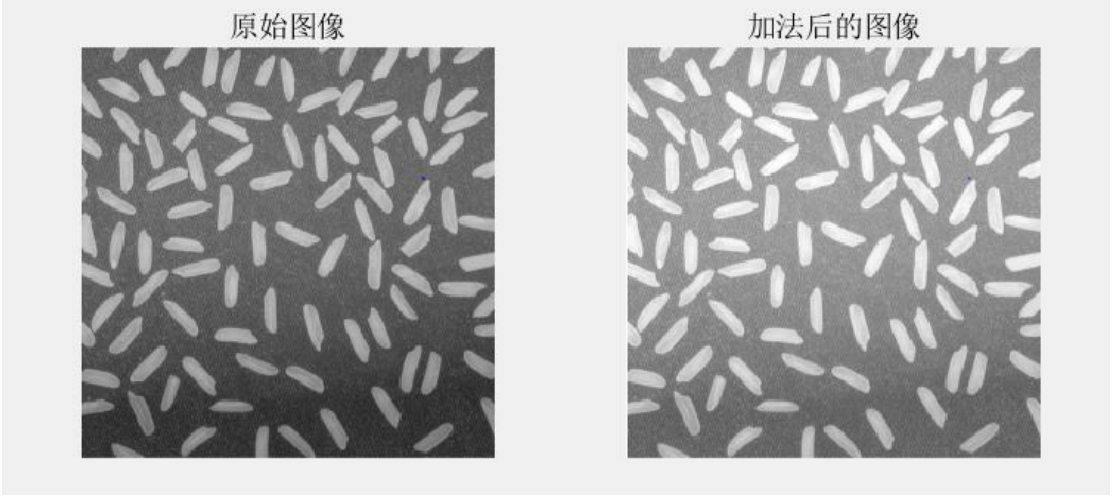
实验总结

imfinfo 函数可以提供图像文件的详细元数据，对于图像处理中的批量图像管理和图像属性获取非常有用。通过了解图像的基本信息，我们可以更好地进行图像预处理、压缩或其他处理操作。

实验七：图像加法操作

实验结果

通过 **imadd** 函数将图像 **rice.png** 与常数值 **50** 相加，结果显示图像亮度明显增强，图像的亮度提高，但细节保留不变。相加后的图像比原图更亮，具有更高的视觉清晰度。



实验总结

本实验展示了图像加法操作的基本应用，**imadd** 函数将常数加到图像的每个像素上，用于增强图像的亮度。这种操作常用于图像亮度调整和图像增强处理。

实验八：两幅图像相加

实验结果

通过 **imadd** 函数将 **rice.png** 和 **cameraman.tif** 两幅图像相加，得到一幅新的图像。相加后的图像显示了两幅图像的特征，像素值进行了累加，使得合成后的图像既保留了原图的细节，又融合了两幅图像的元素。



实验总结

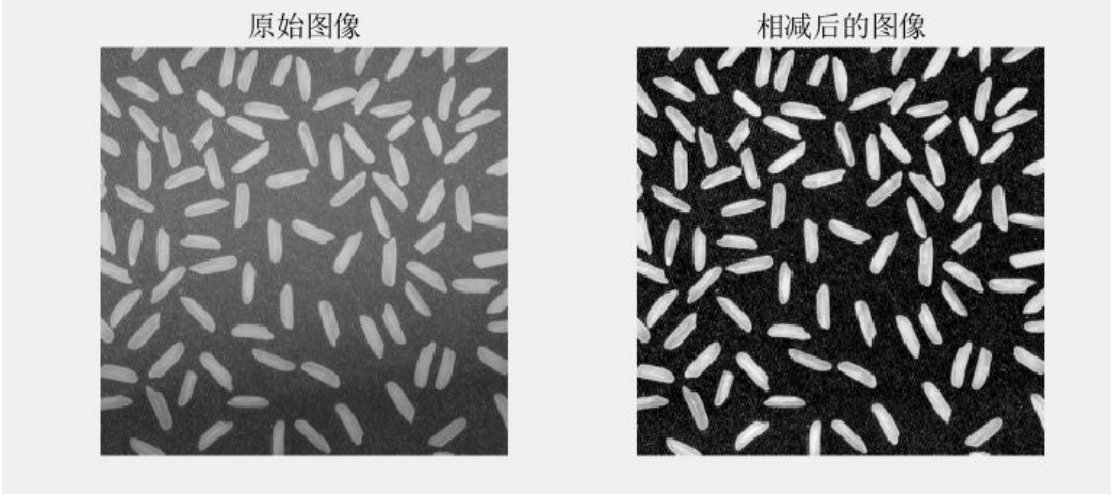
图像相加是一种常见的图像合成技术，可以将两幅图像结合成一幅新的图像。在

实际应用中，图像合成常用于图像融合、增强或特效制作。

实验九：两幅图像相减

实验结果

通过 **imsubtract** 函数对 **rice.png** 图像与背景图像进行相减，成功地提取出了图像中的前景部分。相减后，背景被去除，只保留了图像中的有用信息和前景特征。



实验总结

图像相减操作可以有效去除背景，只保留前景信息，广泛应用于图像差异检测、目标提取等领域。通过 **imsubtract** 函数，可以突出显示图像中感兴趣的部分。

实验十：两幅图像相乘

实验结果

使用 **immultiply** 函数将 **moon.tif** 图像与常数 **116** 相乘，图像亮度明显增强。通过乘法操作，图像的每个像素值都被放大，导致图像变得更加明亮。



实验总结

图像相乘操作是一种亮度增强的方法，可以通过调整乘数来控制图像亮度的增减。在某些应用中，如图像加权处理和亮度调整，相乘操作常用于对图像进行局部或全局的亮度修正。